Sarika Pujari
AMB2223

Advanced Problem-Solving Lab – High-IQ Track

Activity H1 – Solve the "Hidden Problem" Behind the Stated Problem

Title: From Surface Problem to Real Problem

Goal
Train yourself to see the *real* problem, not just the stated one.

Instructions
You are given a "client statement":

"Our exam system is too slow. We want a Python script to speed it up."

1. Identify at least 3 possible *real* problems this statement might hide.
   - Hidden problem 1: Database inefficiency
   - Hidden problem 2: Sensor overload
   - Hidden problem 3: poor code logic
2. For each hidden problem, state:
   - What data would you need to confirm it?
     - Problem 1: Query time
     - Problem 2: sensor usage
     - Problem 3: code profiling
3. Choose ONE hidden problem and re-frame it as a clear technical problem statement (2–3 lines):

New problem statement:

system is slow due to inefficient database queries

4. Design a very small Python experiment that could validate or reject your reframed problem (pseudo-code is fine):

python

 # Experiment idea (what will you measure, on what data, how?)

```
start = time.time()
run_query()
print(time.time() - start)
```

Activity H2 – Multiple Competing Solutions

Title: Design 3 Solutions Before Coding 1

Goal
Avoid fixation. Explore at least three distinct solution strategies.

Instructions
Problem:

"You receive a stream of log lines (text). You must detect 'suspicious' activity in real time."

1. Give three different solution ideas, each using a distinct main technique or data structure.
   - Approach A (e.g., rule-based / regex / thresholds):
     Rule – based
   - Approach B (e.g., statistical / sliding window / counters):
     sliding window
   - Approach C (e.g., simple ML / anomaly detection concept):
     ML model
2. For each approach, list pros and cons (at least one each).
   - Approach A – Pros: Simple Cons: limited
   - Approach B – Pros: Balanced Cons: needs tuning
   - Approach C – Pros: Smart Cons: complex
3. Choose one approach to implement first and justify:

I choose approach R *because* simple & effective

4. Sketch high-level Python structure (modules/classes, not full code):

python

 # Modules / classes / key functions list
functions for reading logs, processing, detecting

Activity H3 – Self-Constrained Challenge: "Elegant Code Only"

Title: Solve with Elegance (Clarity, Minimalism, Correctness)

Goal

Produce code that is not only correct, but also minimal and beautiful.

Problem

Implement a mini "pipeline" for processing a list of tasks:

- Each task has an ID, description, and status ("todo", "doing", "done").
- You must:
  - Load tasks from a JSON file.
  - Filter by status.
  - Sort by ID.
  - Print a clean summary.

Constraints (you impose on yourself)

1. Maximum 25 lines of *real* Python (excluding blanks and comments).
2. No function longer than 7 lines.
3. Names must be meaningful and short.

Steps

1. Design your data model in 2–3 lines:

model : Task (id, dese, status)

2. Decide at least one thing you will *not* do to keep it minimal:

Not doing : Error handling

3. Write your code here (respecting constraints):

python

 # Elegant pipeline solution

```
import json
tasks = json.load(open("tasks.json"))
tasks = [t for t in tasks if
t["status"] == "todo"]
tasks = sorted(tasks, key=lambda
x: x["id"]
for t in tasks:
print(t)
```

4. Reflection:

- What did you have to sacrifice (if anything) to keep elegance?

Skipped validation for simplicity

Activity H4 – Complexity and Scaling Thought Experiment

Title: How Will Your Solution Break at Scale?

Goal
Develop realistic intuition about performance and scaling.

Instructions
Take ANY non-trivial function or algorithm you have recently written (search, summarization, scoring, etc.).

1. Describe what it does, in *one line*:
   Search for a specific item in a list
2. Estimate its time complexity in Big-O notation, and justify in 1–2 lines:
   - Big-O: O(n)
   - Because: it checks each element once
3. Scaling scenarios – imagine input sizes:
   - $n = 10^3$ (1,000)
   - $n = 10^6$ (1,000,000)
   - $n = 10^9$ (1,000,000,000)

For each, predict: will it still be usable (Y/N) and why?

n	Usable (Y/N)	Reason (1 short sentence)
10^3	Yes	very fast
10^6	Yes	Slight delays
10^9	No	too slow

4. Suggest one change to improve scalability and briefly describe how it helps:
   use binary search (O(log n)) if data is sorted → faster performance

Activity H5 – Adversarial Thinking: Break Your Own Design

Title: Attack Your Own Program

Goal

Develop robustness and defensive mindset.

Instructions

Choose one of your own projects or mini-tools (file reader, API client, CLI, etc.).

1. Write a short description of what it is supposed to do:

   A file reads that reads & displays file content

2. Now wear the hat of an *attacker* or malicious user.
   List 5 ways to misuse / break it:

   - Attack 1: upload huge file
   - Attack 2: malicous file path
   - Attack 3: unsupported file format
   - Attack 4: inject special character
   - Attack 5: Repeated requested (DOS)

3. For each attack, suggest one defence in plain language:

   - Defence for attack 1: Limit file size
   - Defence for attack 2: Validate file path
   - Defence for attack 3: Allow only specific formats
   - Defence for attack 4: sanitize input
   - Defence for attack 5: Rate limiting

4. Implement at least one of these defences in your code and paste the updated snippet:

python

 # Hardened code snippet:

```
def read_file(path):
    if not path.endswith(".txt"):
        return "Invalid file type"
    with open(path, "r") as f:
        return f.read(1000) # limit size
```

Activity H6 – Cross-Domain Transfer

Title: Apply Known Pattern to a New Domain

Goal
Practice lifting a known idea/pattern from one domain to a different one.

Instructions
Pick a pattern you already know well, e.g.:

- Sliding window
- Two-pointer technique
- Dynamic programming table
- Producer–consumer
- Map–reduce style
- Pipeline of filters

1. Name of pattern: Sliding window
2. Original domain where you learned it (e.g., arrays, strings):
   Arrays
3. New domain to apply it in (e.g., logs, time series, learning activities, medical data):
   log analysis
4. Describe how you would adapt the pattern to this new domain, in 5–8 lines (can include pseudo-code):

use a time-based window
continuously track events within the window, if suspicious
activity exceeds threshold → trigger alert

5. Optional: implement a small prototype showing this transfer:

python

```
# Pattern transfer prototype
from collections import deque
import time
logs = deque()
def add_event():
    now = time.time()
    logs.append(now)
    while logs and now - logs[0] > 60:
        logs.popleft()
    if len(logs) > 10:
        print("suspicious activity")
```

Activity H7 – Meta-Cognition: Your Personal Problem-Solving Manual

Title: Write Your Own "How I Solve Problems" Guide

Goal

Turn your tacit skills into an explicit, reusable system.

Instructions

1. List the first three things you usually do when you get a new problem (be honest):
   - Step 1: Read problem quickly
   - Step 2: Thinks of solution immediately
   - Step 3: Start coding up
2. Now, design your ideal 6–8 step process (what a "future you" would always do):

1. Understand problem clearly
2. Identity constraints
3. Break into sub-problems
4. Think multiple solutions
5. Choose best approach
6. Plan before coding
7. (optional) —
8. (optional) —

3. For each step, write one question you will ask yourself:
   - Step 1 question: What is the actual problem?
   - Step 2 question: What are the limitations?
   - Step 3 question: Can I divide this problem?
   - Step 4 question: Are there better approaches?
   - Step 5 question: Which is most efficient?
   - Step 6 question: How will I implement it?
4. Final instruction to yourself (one sentence):

"When I feel stuck, I will step back *."* rethink the problem & try a simple approach